# Practical-7

## Problem Definition

Simulate an incremental Change Data Capture (CDC) loop to track and capture all modifications (INSERT, UPDATE, and DELETE operations) occurring in an operational transaction system.

The captured database changes must be recorded in an audit logging ledger and correctly replicated to an analytical table to maintain data consistency without performing destructive full-table database dumps.

This practical focuses on developing an incremental data synchronization pipeline using PostgreSQL, Python, Psycopg2, PL/pgSQL event logs, and message replication loops. The system will simulate inventory updates, new order insertions, and record deletions, then propagate these changes to an analytical database table.

## Dataset

The dataset consists of simulated application inventory database transactions.

The records include:

- Product or inventory IDs.
- Product names and descriptions.
- Available stock quantities.
- Product prices.
- Transaction types (INSERT, UPDATE, DELETE).
- Previous and updated values.
- Event timestamps.
- Transaction or event IDs.
- Audit records and synchronization status.

The test data is used to simulate new inventory records, stock quantity changes, and deleted records. These changes are captured incrementally and propagated to the analytical table.

# Architecture and Workflow

The CDC pipeline works by capturing database changes as they happen and forwarding them to an analytical table.

1. Operational Database

PostgreSQL inventory table

|

2. Change Detection

Capture INSERT, UPDATE and DELETE

|

3. CDC Event Log

Store operation, old values, new values and timestamp

|

4. Python Replication Loop

Read pending events and apply changes

|

5. Analytical Table

Maintain a synchronized copy of inventory data

# Code:

## 1. Create the database tables

```
CREATE TABLE IF NOT EXISTS inventory (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    stock INT,
    price NUMERIC(10, 2)
);


CREATE TABLE IF NOT EXISTS cdc_events (
```

```
    event_id BIGSERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    operation VARCHAR(10) NOT NULL,
    old_data JSONB,
    new_data JSONB,
    event_time TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    processed BOOLEAN DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS analytical_inventory (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    stock INT,
    price NUMERIC(10, 2)
);
```

## 2. Create the CDC trigger function

```
CREATE OR REPLACE FUNCTION capture_inventory_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO cdc_events
            (product_id, operation, old_data, new_data)
        VALUES
            (NEW.product_id, 'INSERT', NULL, to_jsonb(NEW));

        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
```

```
        INSERT INTO cdc_events
            (product_id, operation, old_data, new_data)
        VALUES
            (NEW.product_id, 'UPDATE',
             to_jsonb(OLD), to_jsonb(NEW));

        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO cdc_events
            (product_id, operation, old_data, new_data)
        VALUES
            (OLD.product_id, 'DELETE', to_jsonb(OLD), NULL);

        RETURN OLD;
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

## 3. Attach the trigger to the source table

```
DROP TRIGGER IF EXISTS inventory_cdc_trigger
ON inventory;

CREATE TRIGGER inventory_cdc_trigger
AFTER INSERT OR UPDATE OR DELETE
ON inventory
FOR EACH ROW
```

```
EXECUTE FUNCTION capture_inventory_changes();
```

## 4. Insert sample inventory records

```
-- INSERT operation
INSERT INTO inventory
VALUES
    (101, 'Laptop', 50, 55000.00),
    (102, 'Keyboard', 100, 1200.00);

-- UPDATE operation
UPDATE inventory
SET stock = 45
WHERE product_id = 101;

-- DELETE operation
DELETE FROM inventory
WHERE product_id = 102;
```

## 5. Replicate the captured events using Python

```
import psycopg2
from psycopg2.extras import RealDictCursor

DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "postgres",
    "user": "postgres",
    "password": "YOUR_PASSWORD"
}
```

```
def replicate_cdc_events():
    conn = psycopg2.connect(**DB_CONFIG)

    try:
        with conn:
            with conn.cursor(
                cursor_factory=RealDictCursor
            ) as cur:

                # Lock and read pending events in order.
                cur.execute("""
                    SELECT *
                    FROM cdc_events
                    WHERE processed = FALSE
                    ORDER BY event_id
                    FOR UPDATE
                """)

                events = cur.fetchall()

                for event in events:
                    operation = event["operation"]
                    new_data = event["new_data"]
                    old_data = event["old_data"]

                    if operation in ("INSERT", "UPDATE"):
                        cur.execute("""
```

```
            INSERT INTO analytical_inventory
                (product_id, product_name, stock, price)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (product_id)
            DO UPDATE SET
                product_name = EXCLUDED.product_name,
                stock = EXCLUDED.stock,
                price = EXCLUDED.price
        """, (
            new_data["product_id"],
            new_data["product_name"],
            new_data["stock"],
            new_data["price"]
        ))

    elif operation == "DELETE":
        cur.execute("""
            DELETE FROM analytical_inventory
            WHERE product_id = %s
        """, (
            old_data["product_id"],
        ))

    # Mark the event processed in the same
    # transaction as the analytical change.
    cur.execute("""
        UPDATE cdc_events
        SET processed = TRUE
        WHERE event_id = %s
```

```
            """, (event["event_id"],))

            print(
                f"Event {event['event_id']}: "
                f"{operation} replicated"
            )

    print("CDC replication completed successfully.")

finally:
    conn.close()

replicate_cdc_events()
```

### 6. Verify the analytical table

```
SELECT *
FROM analytical_inventory
ORDER BY product_id;
```

# Key Questions / Analysis / Interpretation

## Question 1: Contrast the background resource footprint of log-based Change Data Capture mechanisms against classic query-based scheduling checks on production tables.

Answer:

Change Data Capture (CDC) captures database modifications incrementally instead of repeatedly scanning the entire source table.

Log-based CDC reads database transaction logs, such as PostgreSQL's Write-Ahead Log (WAL), to identify changes. It generally reduces the need for repeated full-table scans.

Query-based scheduling checks periodically execute queries to identify modified records. These checks may consume additional CPU, memory, and disk I/O, particularly when tables are large.

The major differences include:

- Resource consumption: Log-based CDC can reduce repeated table-scanning overhead.
- Latency: CDC can propagate changes with low latency, while scheduled polling introduces delays between checks.
- Database load: Query-based checks can increase the load on operational tables.
- Change detection: Log-based CDC can capture transaction-level changes, while polling depends on available timestamps, version columns, or other change indicators.
- Implementation complexity: Log-based CDC requires transaction-log integration and checkpoint management.

Therefore, log-based CDC is generally suitable for large, continuously changing databases where low-latency synchronization is required.

## Question 2: How does your downstream replication engine handle network connection failures without losing event order or creating duplicate data writes?

Answer:

A downstream replication engine can use reliable event processing techniques to recover from connection failures and maintain data consistency.

Important techniques include:

- Persistent event logs: Captured changes remain stored until successfully processed.
- Event ordering: Events are processed according to their transaction or event sequence.
- Acknowledgements: Events are marked as processed only after the corresponding analytical changes succeed.
- Retries: Failed operations are retried after the connection is restored.
- Idempotent writes: Upserts and unique event identifiers help prevent duplicate results when an event is replayed.
- Checkpointing: The last successfully processed position is recorded to resume replication.

In the implementation above, the analytical update and the event's processed status are committed in the same PostgreSQL transaction. This prevents an event from being marked

processed if its analytical update fails. A production system also needs appropriate retry logic and safeguards for concurrent consumers.

### Question 3: What role does the transaction Write-Ahead Log (WAL) fill in enabling real-time, low-impact CDC pipelines in production?

Answer:

The Write-Ahead Log (WAL) is a PostgreSQL mechanism that records database changes in a sequential log before the corresponding data pages are written to their final locations.

Its role in CDC includes:

- Change tracking: WAL records provide information about committed database modifications.
- Incremental replication: Consumers can capture changes without repeatedly copying the entire table.
- Recovery: Log positions allow a consumer to resume from a known checkpoint, provided the required WAL records remain available.
- Transaction ordering: Log sequence numbers (LSNs) help establish the order of database changes.
- Replication support: PostgreSQL logical decoding can convert WAL records into a stream of logical row changes.

WAL-based CDC can reduce the overhead of repeatedly querying production tables. However, it is not entirely zero-impact: log retention, decoding, replication, and network activity still consume resources.

# Supplementary Problem

**Problem statement: Implement downstream deduplication logic that handles out-of-order CDC updates by tracking database row version numbers.**

Solution approach:

1. Add a version number to each source inventory record.
2. Increment the version number whenever the record is updated.
3. Include the version number in each captured event.
4. Store the last applied version in the analytical table.
5. Apply an incoming event only if its version is newer than the version already stored.

6. Record duplicate or outdated events for auditing.

For example, if version 5 is already applied and version 4 arrives late, the replication engine should reject version 4 rather than overwrite the newer state.

This approach helps maintain the consistency of analytical data when events are delayed, duplicated, or delivered out of order.

# Output

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> Get-ChildItem -Recurse -Depth 2
    Directory: C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        28-09-2026     13:30                results
d-----        28-09-2026     13:30                screenshots
d-----        28-09-2026     13:20                scripts
d-----        28-09-2026     13:20                sql
-a----        28-09-2026     13:15           1017 docker-compose.yml
-a----        28-09-2026     13:38          10903 README.md
-a----        28-09-2026     13:39          30838 REPORT_NOTES.md
-a----        28-09-2026     13:15             61 requirements.txt
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> docker compose up -d; docker compose ps
Container operational-db Running
Container analytical-db Running
NAME            IMAGE              COMMAND                  STATUS                  PORTS
analytical-db   postgres:16-alpine  "docker-entrypoint.s…"   Up 3 hours (healthy)    0.0.0.0:5433->5432/to
operational-db  postgres:16-alpine  "docker-entrypoint.s…"   Up 3 hours (healthy)    0.0.0.0:5432->5432/to
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/setup_databases.py
============================================================
TASK 1: DATABASE CONNECTION & SETUP
Operational DB:  Host: localhost | Port: 5432 | Status: PASS
Analytical DB:   Host: localhost | Port: 5433 | Status: PASS
CDC Trigger Check: PASS (trg_capture_inventory_cdc active)
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/seed_database.py
============================================================
TASK 5: INITIAL INVENTORY DATA SEEDING
Inserted 25 inventory records across 5 categories.
Initial Inventory Record Count:  25
CDC Events Generated by Trigger: 25
Event Operation Types Captured:  25 INSERT, 0 UPDATE, 0 DELETE
Trigger CDC Verification:        PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/cdc_capture.py
============================================================
TASK 6: CDC EVENT LEDGER AUDIT
Event ID | Operation | Product ID | Version | Timestamp
       1 | INSERT    |        101 |       1 | 2026-09-28 07:18:14.287
       2 | INSERT    |        102 |       1 | 2026-09-28 07:18:14.290
       3 | INSERT    |        103 |       1 | 2026-09-28 07:18:14.292
       4 | INSERT    |        104 |       1 | 2026-09-28 07:18:14.294
       5 | INSERT    |        105 |       1 | 2026-09-28 07:18:14.296
       ... [20 additional records in ledger] ...
Total CDC Events in Operational Ledger: 25 (Unprocessed: 25)
CDC Event Ledger Status: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/simulate_changes.py
============================================================
TASK 7: SIMULATING LIVE TRANSACTIONAL MUTATIONS
1. INSERT Product 201 (Gaming Laptop 16-inch)     -> COMMITTED
2. INSERT Product 202 (Mechanical Numpad RGB)     -> COMMITTED
3. UPDATE Product 101 (Laptop 15.6-inch) (ver=2)  -> COMMITTED
4. UPDATE Product 102 (Keyboard Mech)    (ver=2)  -> COMMITTED
5. DELETE Product 103 (Mouse Wireless)   (ver=1)  -> COMMITTED (Tombstone)
Total Operational CDC Events Generated: 30
Simulation Status: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/replicate_changes.py
------------------------------------------------------------
TASK 8: CDC INCREMENTAL REPLICATION
Events Fetched from Ledger: 30
  INSERT Events:            27
  UPDATE Events:            2
  DELETE Events:            1
Successfully Replicated:    30
Failed Replications:        0
Skipped (Deduplicated):     0
Operational Ledger Update:  Marked 30 events as processed = TRUE
Replication Status:         PASS (Downstream Analytical DB Synchronized)
------------------------------------------------------------
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/deduplication.py
============================================================
TASK 9: DOWNSTREAM VERSION-BASED DEDUPLICATION & OUT-OF-ORDER CHECK
Test Product ID: 999
  Scenario 1: Version 5 arrives           -> APPLIED (Target ver=5)
  Scenario 2: Version 4 arrives afterward -> REJECTED / SKIPPED (Stale ver)
  Scenario 3: Version 5 duplicate arrives -> REJECTED / SKIPPED (Duplicate)
  Scenario 4: Version 6 arrives           -> APPLIED (Target ver=6)
Deduplication Status: PASS (Analytical Consistency Protected)
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/failure_simulation.py
============================================================
TASK 10: NETWORK / DOWNSTREAM SERVICE FAILURE SIMULATION
Connection Failure Simulated:   YES (Target port 5999 unreachable)
Events Retained in CDC Ledger:  YES (2 pending events buffered)
Events Lost:                    0
Replication Resumed:            YES (Caught up cleanly on port 5433)
Duplicate Writes Detected:      0
Recovery Status:                PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/verify_replication.py
============================================================
TASK 11: FINAL REPLICATION VERIFICATION & PARITY AUDIT
Operational Row Count:       27
Analytical Row Count:        27
CDC Events Captured:         32
CDC Events Processed:        32
Version Consistency:         PASS
Data Consistency:            PASS (0 mismatched rows)
Delete Propagation Check:    PASS (Product 103 purged)
Deduplication Protection:    PASS
Event Ordering:              PASS (Strict event_id monotonic sequence)
No Event Loss:               PASS (32/32 processed)
Overall CDC Verification:    PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_7> python scripts/run_all.py
============================================================
DATA ENGINEERING LABORATORY — PRACTICAL 7 MASTER PIPELINE
Step 1: Database Setup & Triggers Verification... [PASS]
Step 2: Operational Data Seeding (25 records)...   [PASS]
Step 3: CDC Ledger Inspection...                   [PASS]
Step 4: Live Transaction Changes Simulation...     [PASS]
Step 5: Python Incremental Replication...          [PASS]
Step 6: Version Deduplication & Ordering Test...   [PASS]
Step 7: Resilience & Failure Recovery Test...      [PASS]
Step 8: End-to-End Parity Verification...         [PASS]
ALL PRACTICAL 7 TASKS EXECUTED SUCCESSFULLY — STATUS: PASS
============================================================
```

# Final Conclusion

The practical demonstrates how Change Data Capture (CDC) enables incremental synchronization between operational and analytical databases. By capturing INSERT, UPDATE, and DELETE operations, the system maintains an audit ledger and propagates changes to the analytical table without repeatedly copying the complete source database.

PostgreSQL triggers, PL/pgSQL, Python, and Psycopg2 provide a practical foundation for demonstrating CDC concepts. The event log preserves change history, while the replication process updates the analytical table to reflect the latest operational state.

Overall, CDC helps reduce unnecessary data processing, supports timely analytical updates, and provides a foundation for reliable database replication, auditing, and real-time data engineering pipelines.